概要設計

作成日	2026年10月6日 13:58
科目名	プログラミング言語

画像編集ツール 概要設計

1. 文書の位置づけ

本書は、要求分析および要件分析で定義した内容をもとに、実装へ進むためのシステム構成・画面構成・主要モジュール・データ構造・処理フロー・担当分担・技術検証項目を定義する。

参照：要求分析、要件分析

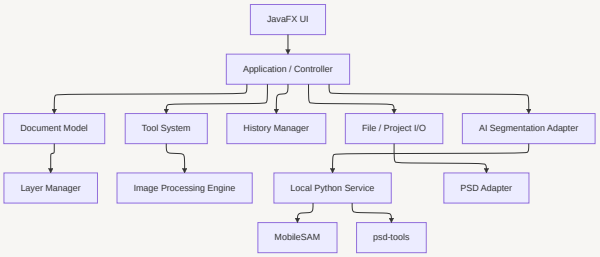
---

2. 設計方針

- メインアプリケーションはJavaで実装し、GUIにはJavaFXを用いる。
- GUI、画像処理、レイヤー管理、履歴管理、ファイル入出力、AI処理を分離する。
- 画像編集処理は可能な限りGUIに依存させず、単体テスト可能な構造とする。
- AI処理はローカルPythonサービスとして実行し、Java側からlocalhost HTTPで呼び出す。MobileSAMを条件付き正式採用とし、最大3候補の提示とMPS失敗時のCPUフォールバックを実装条件とする。
- PSDは完全互換を目標とせず、読み込めたレイヤーを内部共通形式へ変換して扱う。
- 7週間のため、MUSTを先に完成させ、SHOULD、ADVANCEDの順に追加する。

---

3. 全体アーキテクチャ



3.1 UI層

画面表示、入力受付、ダイアログ表示を担当する。画像処理ロジックを直接持たない。

3.2 アプリケーション層

UIからの操作を受け、Document、Tool、History、AI、File機能を連携させる。

3.3 ドメイン／編集モデル層

編集中画像、レイヤー、キャンバスサイズ、選択状態、変更状態などを保持する。

3.4 画像処理層

回転、反転、リサイズ、色調整、ぼかし、モザイク、フィルターを担当する。

3.5 AI連携層

画像と指定位置等をAIへ渡し、返却されたマスクをレイヤーへ変換する。

3.6 永続化層

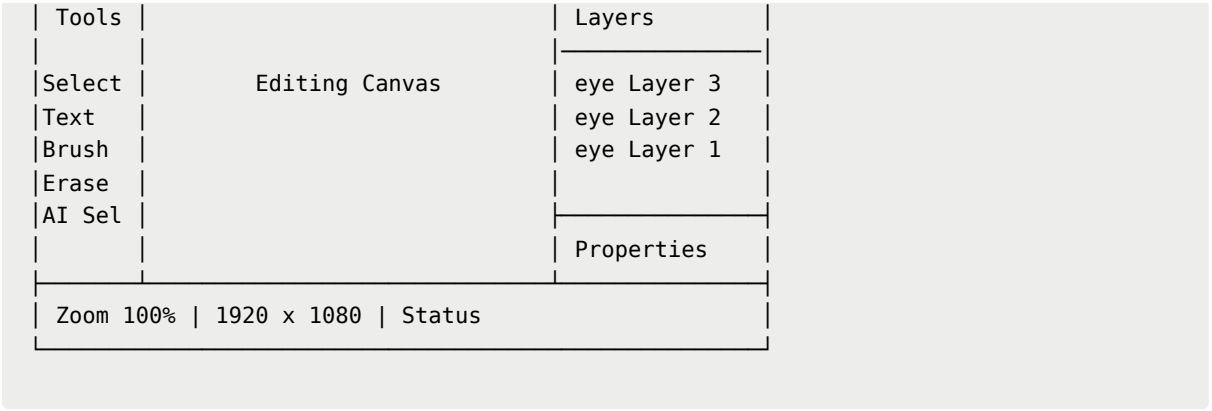
PNG/JPEG、独自プロジェクト、PSDの読み書きを担当する。

---

4. 画面構成

4.1 メイン編集画面

```
┌──────────────────────────────────────────────────────┐
│ File Edit Image Filter AI                            │
├──────┬──────────────────────────────────┬────────────┤
```



主な領域

- メニューバー：開く、保存、書き出し、Undo／Redo等
- ツールバー：選択、文字、ペン、消しゴム、AI選択等
- 編集キャンバス：画像・レイヤーを合成表示
- レイヤーパネル：レイヤー一覧、表示、順序、名称変更
- プロパティパネル：透明度、位置、文字設定など
- ステータスバー：ズーム率、画像サイズ、処理状態

4.2 画像サイズ変更ダイアログ

- 幅
- 高さ
- 縦横比維持
- 確定／キャンセル

4.3 色調整ダイアログ

- 明るさ
- コントラスト
- 彩度
- プレビュー
- 適用／キャンセル

4.4 AI選択画面／モード

キャンバス上で対象をクリックし、AIが返した最大3候補を半透明マスクとして切り替え表示する。最高スコア候補を自動確定しない。

- 候補1〜3の選択
- 正クリック／負クリックによる追加補正
- 確定
- 再試行
- キャンセル

を提供する。

4.5 PSD読み込み警告

未対応要素が存在した場合、再現できない要素を一覧または要約して通知する。

---

5. 主要ユースケース

UC-01 画像を開いて編集・保存する

1. PNG/JPEGを開く
2. Documentを生成する
3. 基本編集を行う
4. Undo／Redoを必要に応じて行う
5. PNG/JPEGへ書き出す

UC-02 レイヤー編集

1. レイヤーを追加する
2. 選択する
3. 移動、透明度変更、順序変更等を行う
4. 合成結果をキャンバスに反映する

UC-03 AIレイヤー分離

1. AI選択モードにする

2. 対象画像をAIサービスへ登録し、画像埋め込みをキャッシュする
3. 対象物をクリックする
4. クリック列をAIへ送信し、最大3候補のマスクを取得する
5. 候補を切り替えてプレビューする
6. 必要に応じて正・負の追加クリックで補正する
7. 確定時に透明背景のRasterLayerを生成する
8. 生成操作をHistoryへ登録する

UC-04 PSD読み込み

1. PSDを指定する
2. PSD Parserで読み込む
3. 対応レイヤーを内部Layerへ変換する
4. 未対応要素があれば通知する
5. 通常レイヤーとして編集する

UC-05 未保存状態で終了する

1. 変更済みDocumentで終了操作を行う
2. 保存／破棄／キャンセルを表示する
3. 選択に従って処理する

---

6. クラス・モジュール構成案

6.1 UI

- `MainWindowController`
- `CanvasController`
- `LayerPanelController`
- `ToolBarController`
- `PropertyPanelController`
- `DialogService`

6.2 編集モデル

- `ImageDocument`
- `CanvasState`
- `LayerManager`
- `Layer`
- `RasterLayer`
- `TextLayer`
- `DrawingLayer`

6.3 ツール

```
Tool
├─ SelectTool
├─ CropTool
├─ BrushTool
├─ EraserTool
├─ TextTool
└─ AiSelectTool
```

`Tool`インターフェースを共通化し、マウス入力等を各ツールへ委譲する。

6.4 画像処理

- `ImageProcessor`
- `ResizeProcessor`
- `RotateProcessor`
- `ColorAdjustProcessor`
- `BlurProcessor`
- `MosaicProcessor`
- `FilterProcessor`

フィルターは以下のような共通インターフェースを想定する。

```
interface ImageFilter {
    BufferedImage apply(BufferedImage source);
```

```
}
```

6.5 履歴管理

- `Command`
- `HistoryManager`
- `AddLayerCommand`
- `DeleteLayerCommand`
- `MoveLayerCommand`
- `ImageEditCommand`

Commandパターンを利用し、Undo／Redoの対象を統一する。移動・透明度・順序変更等は状態値のみを履歴化する。描画やフィルター等のラスタ変更は、変更領域を最大256×256pxの差分タイルへ分割して変更前データを保持する。決定的に再計算できる処理は変更前データと処理パラメータを保持してRedo時に再計算する。履歴上限は実データ量基準で256MiBとし、超過時は古いCommandから破棄する。

6.6 ファイル入出力

- `ImageFileService`
- `ProjectFileService`
- `PsdImportService`
- `ExportService`

6.7 AI

- `AiSegmentationService`
- `AiRequest`
- `AiMaskResult`
- `LayerExtractor`

AI実装方式が変わってもJava側の呼び出しを変更しなくて済むよう、AI処理をインターフェース化する。

---

7. 主要データ構造

7.1 ImageDocument

- canvasWidth
- canvasHeight
- layers
- activeLayerId
- filePath
- dirty

7.2 Layer

共通属性：

- id
- name
- visible
- opacity
- x / y
- rotation
- scaleX / scaleY

7.3 RasterLayer

- BufferedImage image

7.4 TextLayer

- text
- fontFamily
- fontSize
- color

7.5 DrawingLayer

- 描画結果またはストローク情報

7.6 AiMaskResult

- candidates: List<MaskCandidate>
- sourcePoints
- selectedCandidateId

7.7 MaskCandidate

- candidateId
- mask
- score

---

8. プロジェクト保存形式案

SHOULD機能として、独自プロジェクト形式をZIPコンテナ方式で設計する。

```
project.jimg
├─ project.json
├─ layers/
│  ├─ layer-001.png
│  ├─ layer-002.png
│  └─ ...
└─ assets/
```

`project.json`にはキャンバス情報、レイヤー順、位置、透明度、種類、テキスト情報等を保存する。

利点：

- JSONだけより画像データを確実に保持できる
- 元画像の移動・削除に影響されにくい
- Java標準のZIP APIで扱いやすい

---

9. AI連携方式

9.1 採用方式

Javaアプリケーション + localhost限定のローカルPythonサービス構成を正式採用する。AIモデルはMobileSAMを条件付き採用とする。条件は、最大3候補からの選択UIを設けること、MPS推論失敗時にCPUへ切り替えて1回だけ再試行すること、モデル常駐と画像埋め込みキャッシュを行うことである。

```
JavaFX
  ↓ HTTP / localhost
AI Adapter
  ↓
Local Python Service
  ↓
MobileSAM / PyTorch MPS
```

入力

- 編集対象画像
- クリック座標
- 必要に応じて矩形等

出力

- バイナリマスクまたは透過PNG
- 必要に応じて信頼度

9.2 代替案

授業要件がJavaのみの場合は別途再設計する。ONNX Runtime Javaの公式配布物はmacOS ARM64対応が弱いため、直接推論は現時点の本線としない。

9.3 AI失敗時

- タイムアウト
- モデル未起動
- 認識結果なし
- 不正な応答

を区別してエラー表示し、Documentへ変更を加えない。

---

10. PSD設計方針

PSDをアプリ内部で直接編集し続けるのではなく、ローカルPythonサービスの`psd-tools`で解析し、読み込み時に内部のLayerモデルへ変換する。Javaのみ制約の場合はTwelveMonkeys ImageIOを代替候補とする。

```
PSD
 ↓
PsdImportService
 ↓
Internal Layer Model
 ↓
Editor
```

対応優先度：

1. 通常ラスターレイヤー
2. レイヤー名・表示状態・位置・透明度
3. テキストレイヤー（ライブラリ対応状況次第）

対象外候補：

- 高度なレイヤー効果
- スマートオブジェクト完全再現
- 調整レイヤー完全再現
- 複雑な描画モード

---

11. 画像描画・レンダリング方式

キャンバス表示時はLayerManagerが下層から上層へレイヤーを合成する。

```
Layer 1
   ↓
Layer 2
   ↓
Layer 3
   ↓
Composite Preview
```

編集画面では元データを可能な限り保持し、PNG/JPEG書き出し時に最終合成する。ラスターレイヤーの基準表現は`BufferedImage`とし、JavaFX側には表示用

`WritableImage`をキャッシュする。`SwingFXUtils`による全画面変換をマウス移動等の高頻度処理では行わない。

---

12. エラー処理方針

共通エラーとして以下を扱う。

- 未対応画像形式
- 破損画像
- 保存先への書き込み失敗
- メモリ不足
- PSD解析失敗
- AI起動失敗
- AIタイムアウト
- AI認識失敗

エラー時は可能な限り現在の編集状態を保持し、アプリケーションを終了させない。

---

13. 非同期処理

UIスレッドをブロックしないよう、以下はバックグラウンド処理とする。

- AI推論
- PSD読み込み
- 大きな画像のフィルター処理
- プロジェクト保存／読み込み
- 発展機能の一括処理

JavaFXの`Task` / `Service`等を利用し、進捗または処理中表示を行う。

---

14. 2人開発の担当分け

2人が同じクラスを頻繁に編集して衝突することを避けるため、**担当A＝エディタ本体・UI・状態管理、担当B＝画像処理・AI・PSD**を基本境界とする。両者ともJava

コードを担当し、Pythonだけを一方へ偏らせない。週別の作業・受け渡し境界・Git運用は開発分担・進行計画に定義する。

担当	主担当	主な所有領域
A	エディタ本体・UI・状態管理・統合	JavaFX画面、Canvas／Tool操作、ImageDocument、Layer／LayerManager、HistoryManager、256×256差分タイル履歴、PNG/JPEG入出力、.jimg保存、未保存確認、AI候補選択UI、最終統合
B	画像処理・AI・PSD・外部連携	回転・反転・リサイズ・色調整・ぼかし・モザイク・フィルター、ImageFilter群、MobileSAMローカルサービス、Java HTTPクライアント、画像埋め込みキャッシュ、MPS→CPUフォールバック、psd-tools解析、PSD中間DTO、重い処理の非同期化

14.1 境界ルール

- AはUIイベント、Document／Layer状態、履歴、保存状態を所有する。
- Bは画像を入力して結果を返す純粋な画像処理と、AI／PSD外部サービスを所有する。
- Crop、ぼかし、色調整等は、Aが座標・パラメータを取得し、BのProcessorへ渡す。
- AIはBが最大3候補を返すまでを担当し、候補の見せ方・選択・確定によるLayer生成はAが担当する。
- PSDはBが中間DTOとRGBAを返し、Aが内部Layerへ登録してUIへ表示する。
- `Layer`、`Command`、`ImageFilter`、AI／PSD DTOなど両者が利用する契約を変更する場合は、実装前に2人で確認する。

14.2 共同作業

- 要件・インターフェース変更の確認
- 相互コードレビュー
- 週末の結合テスト
- MUST機能の受入テスト
- デモ手順・発表資料の最終確認

15. 開発ブランチ・結合方針

- Gitを使用する。
- `main`は動作確認済み状態を維持する。
- 機能単位のfeature branchを作成する。
- 結合前に最低限ビルド・基本操作テストを行う。
- 大きな共通クラス変更は先にチームで共有する。

例：

```
main
├─ feature/ui
├─ feature/layer
├─ feature/image-processing
├─ feature/ai
└─ feature/psd
```

---

16. 7週間の開発計画

第1週：設計・技術検証

- JavaFXプロジェクト作成
- Git環境作成
- AIモデル動作検証
- PSDライブラリ検証
- Layerモデルの確定

第2週：編集基盤

- メイン画面
- 画像読み込み・書き出し
- キャンバス
- LayerManager
- 基本Undo／Redo

第3週：基本画像編集

- トリミング
- 回転・反転
- サイズ変更
- 文字
- 描画・消しゴム

第4週：画像処理・レイヤー完成

- 色調整
- フィルター
- ぼかし
- モザイク
- レイヤーUI・透明度・移動

第5週：AI統合

- AI選択
- マスクプレビュー
- レイヤー生成
- エラー処理

第6週：SHOULD・結合

- PSD
- プロジェクト保存
- ズーム
- バグ修正
- 結合テスト

第7週：安定化・発表

- 重要バグ修正

- デモ手順固定
- テスト
- 発表資料
- 余裕があればADVANCED機能

---

17. 技術検証項目（完了記録）

一次資料調査は完了済み。詳細は技術検証メモを参照する。PoC-A〜Dはすべて完了済みであり、以下は検証時の確認項目として残す。最終採否と実測結果は技術検証メモおよび第20章を参照する。

TV-01 AIモデル

- 第一候補：MobileSAM
- MPS実行可否、CPUフォールバック、初回解析時間、クリック後のマスク生成時間を実機確認する
- 品質不足時のみSAM 2.1 Tiny、EfficientSAMを比較する

TV-02 PSD

- 第一候補：psd-tools（Python、MIT）
- ラスターレイヤー、グループ、テキスト／調整レイヤーを含む3種類のPSDで互換性を確認する
- Javaのみ制約時の代替候補：TwelveMonkeys ImageIO

TV-03 JavaFX画像処理

- `BufferedImage`を基準表現として2K／4K画像を計測する
- 表示用`WritableImage`キャッシュの更新コストを確認する
- 必要に応じて`PixelBuffer`方式へ最適化する
- 透過PNGが正常に扱えるか確認する

TV-04 Undo／Redo

- Command＋差分スナップショット方式を採用する

- 2K／4K画像で30操作程度を行い、256MiBの履歴予算で実用的なUndo回数を確保できるか確認する

---

18. テスト観点

単体テスト

- 回転・反転結果
- サイズ変更
- 色調整
- フィルター
- LayerManager
- プロジェクト保存・復元

結合テスト

- UIから各編集処理を呼び出せるか
- AI結果からレイヤーを作成できるか
- Undo／Redo後もレイヤー構造が壊れないか
- PNG透明度が保持されるか
- 未保存確認が正しく表示されるか

デモ受入テスト

要求・要件分析で定義した最低完成フローを最初から最後まで実行する。

---

19. 今後詳細設計で確定する項目

技術検証により、AIモデル、PSDライブラリ、画像表現、Undo／Redo保持方式は確定済みとする。詳細設計では以下を確定する。

- localhost HTTPの具体的なJSON／multipartスキーマ、タイムアウト、サービス起動・終了手順
- `.jimg`のJSONスキーマとバージョン管理

- 各クラスのメソッド・責務・例外型
- 256×256差分タイルのクラス構造と履歴破棄条件
- AI候補選択UIの状態遷移
- 画面部品の詳細配置
- ショートカットキー
- 具体的な単体・結合・受入テストケース

20. 技術検証反映（2026-10-06）

実機PoCの合格により、次の項目を概要設計の正式方針として確定する。第17章の技術検証は完了扱いとし、第19章のうちAIモデル、PSDライブラリ、Undo / Redo保持方式は確定済みとする。

20.1 AI連携の確定差分

- `AiSegmentationService` はモデルをプロセス内に常駐させ、画像埋め込みを `imageId` 単位でキャッシュする
- デバイスはMPSを優先し、起動時または推論時に失敗した場合はCPUへ切り替えて1回だけ再試行する
- `/segment` は最高スコア1件ではなく、最大3件の候補マスク、各スコア、候補IDを返す
- UIは候補プレビューの選択と、正クリック・負クリックによる補正を提供する
- `AiMaskResult` は単一 `mask` ではなく `candidates: List` を持つ。`MaskCandidate` は `mask` 、 `score` 、 `candidateId` を持つ
- 初回待ち時間を抑えるため、サービス起動後にダミー推論を1回行う

UC-03は「画像登録・埋め込み計算 → クリック列送信 → 最大3候補取得 → 候補選択または追加クリック → 透明RasterLayer生成 → History登録」の順とする。

20.2 PSD取り込みの確定差分

`psd-tools` を正式採用する。レイヤー名、階層、位置、表示状態、実効表示状態、透明度、合成モード、種別を中間DTOへ保持し、編集画像はRGBAへ抽出して `RasterLayer` へ変換する。TypeLayerとSmartObjectLayerもVersion 1では編集可能なラスターデータとして扱う。未対応の調整・効果はフラット化し、読み込み警告へ列挙する。PSDへの完全再保存は対象外。

20.3 画像表現の確定差分

ラスターデータの正本は`BufferedImage.TYPE_INT_ARGB_PRE`とし、表示側は同じ`WritableImage`を再利用する。4K全体フィルター、PSD解析、保存、AI通信はJavaFX Application Thread外で実行する。`PixelBuffer`は性能問題が確認された場合だけ導入する。

20.4 履歴設計の確定差分

ラスタ変更は最大256×256の差分タイルで保持し、タイルごとに変更前データを保存する。Redoを決定的に再計算できる処理は、変更前データと処理パラメータを保持する。移動、表示、透明度、名称、順序等は値だけを保持する。履歴予算は実データ量基準で256MiBとし、超過時は古いCommandから破棄する。全面4Kスナップショットは使用しない。

20.5 詳細設計へ残す項目

- localhost HTTPの具体的なJSON / multipartスキーマとタイムアウト値
- `.jimg`のJSONスキーマ
- 各クラスのメソッド、例外型、ライフサイクル
- 画面部品、ショートカット、具体的テストケース

以上を前提として詳細設計と実装へ進む。